

# Digital Design (SET 111)

## L08

### Combinational Logic Analysis

Dr. Ahmed Reda Mohamed<sup>1</sup>

<sup>1</sup>PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.



- ☐ Implementation using Logic Gates
- ☐ Implementation using Universal Logic Gates



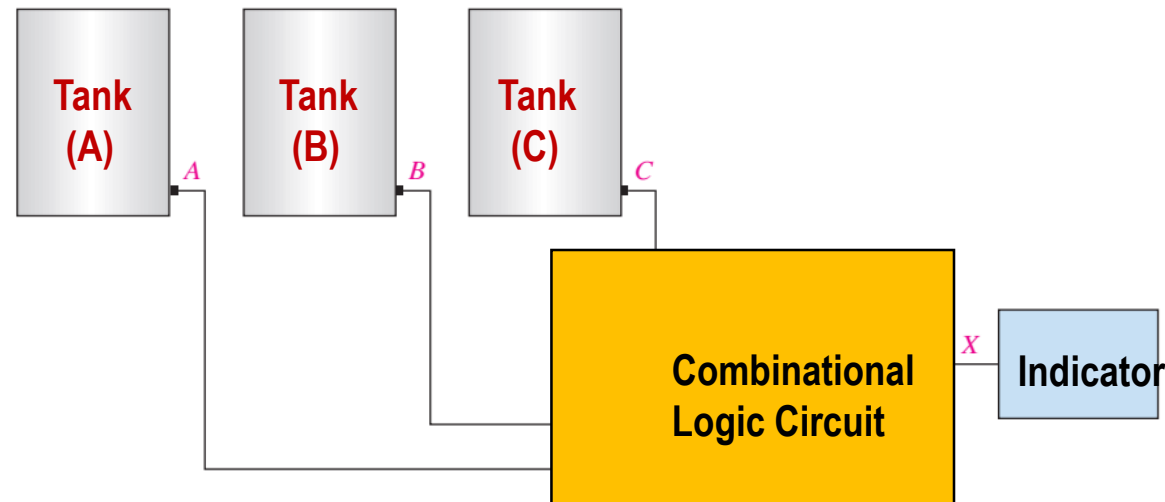
- ❑ Implementation using Logic Gates

- ❑ Implementation using Universal Logic Gates

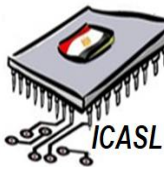
# Implementation using Logic Gates



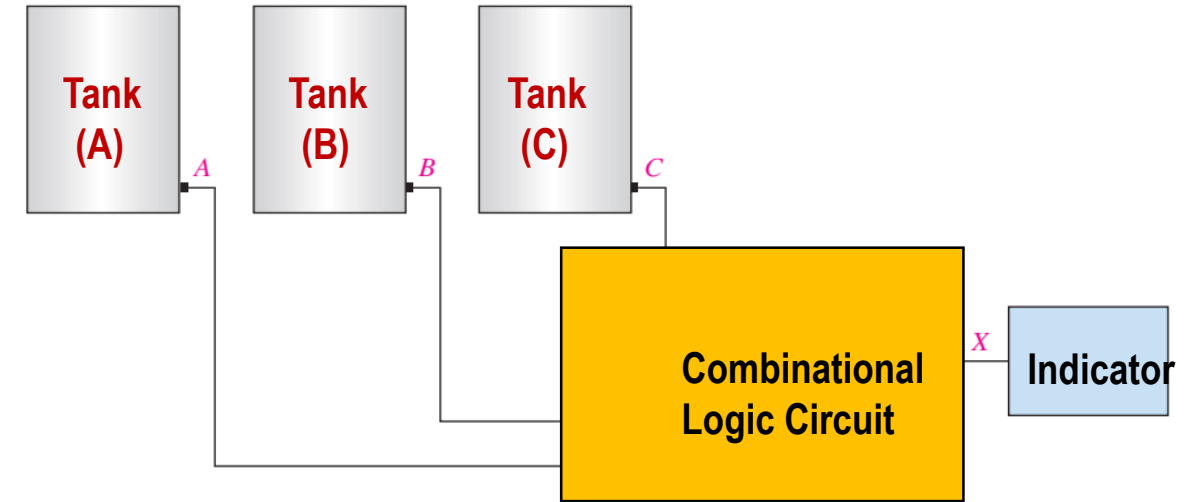
- Ex.1 In a certain chemical-processing plant, a liquid chemical is used in a manufacturing process. The chemical is stored in **three** different tanks. A level sensor in each tank produces a **HIGH** voltage when the level of chemical in the tank drops below a specified point. Design a circuit that monitors the chemical level in each tank and indicates when the level in **any two** of the tanks drops below the specified point.



# Implementation using Logic Gates



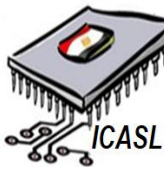
**Ans.**



| Input |   |   | Output |
|-------|---|---|--------|
| A     | B | C | X      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 0      |
| 0     | 1 | 0 | 0      |
| 0     | 1 | 1 | 1      |
| 1     | 0 | 0 | 0      |
| 1     | 0 | 1 | 1      |
| 1     | 1 | 0 | 1      |
| 1     | 1 | 1 | 1      |

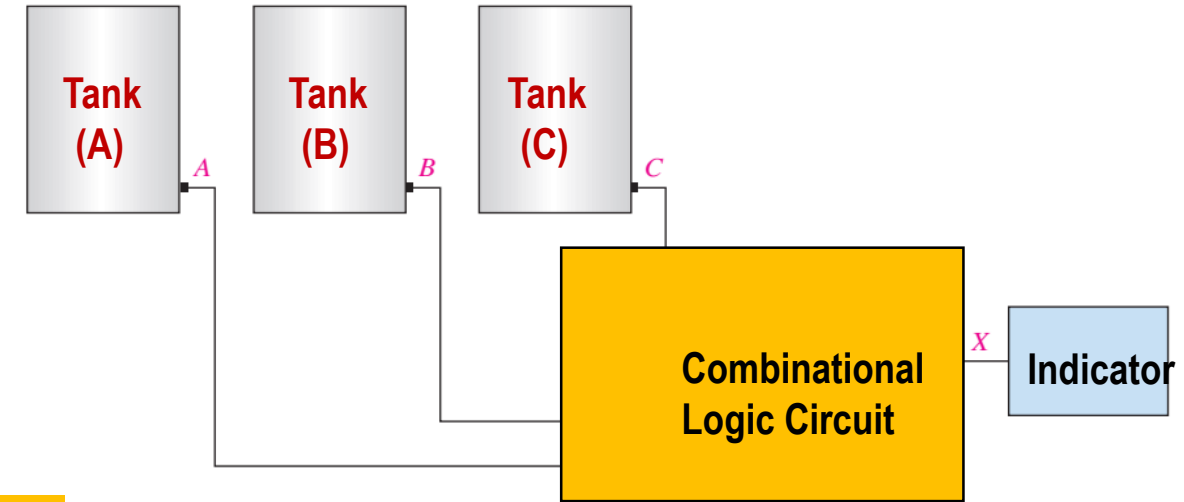
|   |   |    |    |    |    |
|---|---|----|----|----|----|
|   |   | B  |    |    |    |
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 0  | 0  | 1  | 0  |
|   | 1 | 1  | 1  | 1  | 1  |
|   |   | C  |    |    |    |

# Implementation using Logic Gates



**Ans.**

$$X = AB + AC + BC$$



| Input |   |   | Output |
|-------|---|---|--------|
| A     | B | C | X      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 0      |
| 0     | 1 | 0 | 0      |
| 0     | 1 | 1 | 1      |
| 1     | 0 | 0 | 0      |
| 1     | 0 | 1 | 1      |
| 1     | 1 | 0 | 1      |
| 1     | 1 | 1 | 1      |

|   |   | B  |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 0  | 0  | 1  | 0  |
|   | 1 | 1  | 1  | 1  | 1  |

# Implementation using Logic Gates



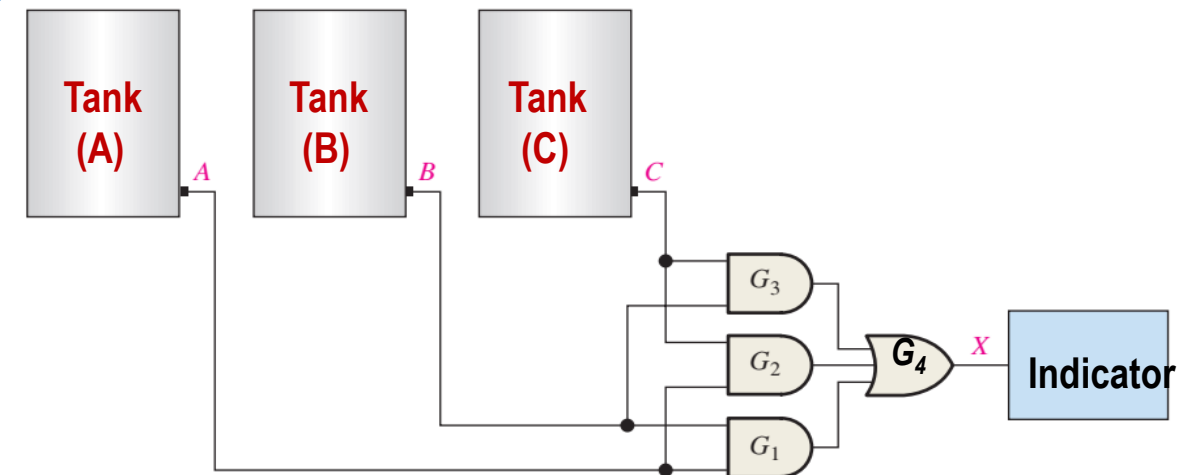
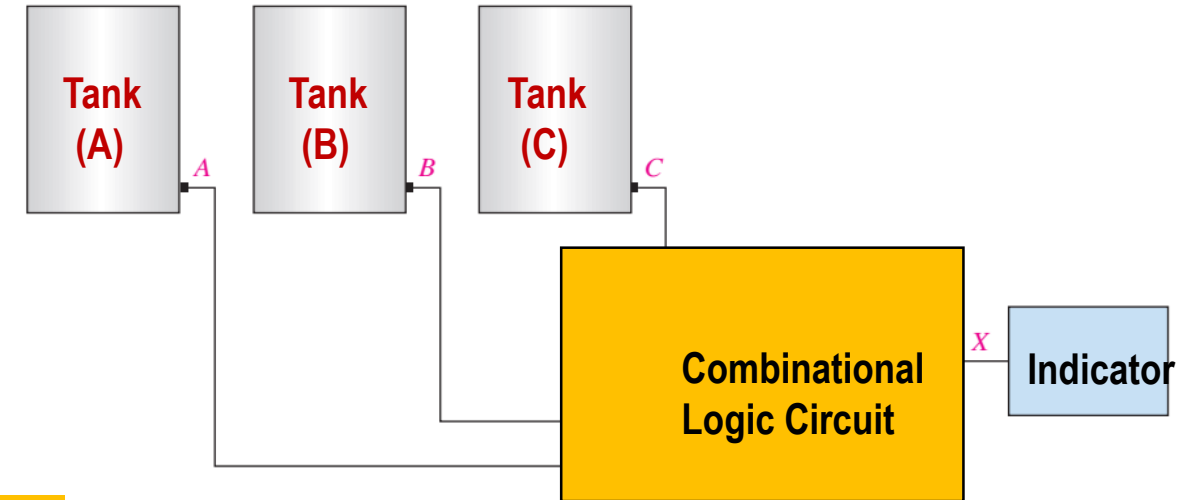
**Ans.**

| Input |   |   | Output |
|-------|---|---|--------|
| A     | B | C | X      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 0      |
| 0     | 1 | 0 | 0      |
| 0     | 1 | 1 | 1      |
| 1     | 0 | 0 | 0      |
| 1     | 0 | 1 | 1      |
| 1     | 1 | 0 | 1      |
| 1     | 1 | 1 | 1      |

|   |   | B  |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 0  | 0  | 1  | 0  |
|   | 1 | 0  | 1  | 1  | 1  |

$$X = AB + AC + BC$$

$G_1$     $G_2$     $G_3$   
 $G_4$



# Implementation using Logic Gates



□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

# Implementation using Logic Gates



□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

| Input |   | Output |
|-------|---|--------|
| A     | B | X      |
| 0     | 0 | 0      |
| 0     | 1 | 1      |
| 1     | 0 | 1      |
| 1     | 1 | 0      |

|          |   | <i>B</i> |   |
|----------|---|----------|---|
|          |   | 0        | 1 |
| <i>A</i> | 0 | 0        | 1 |
|          | 1 | 1        | 0 |

# Implementation using Logic Gates



□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

| Input |   | Output |
|-------|---|--------|
| A     | B | X      |
| 0     | 0 | 0      |
| 0     | 1 | 1      |
| 1     | 0 | 1      |
| 1     | 1 | 0      |

$$X = A \cdot \overline{B} + \overline{A} \cdot B$$

|          |   | <i>B</i> |   |
|----------|---|----------|---|
|          |   | 0        | 1 |
| <i>A</i> | 0 | 0        | 1 |
|          | 1 | 1        | 0 |

# Implementation using Logic Gates



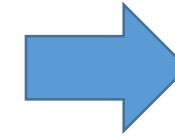
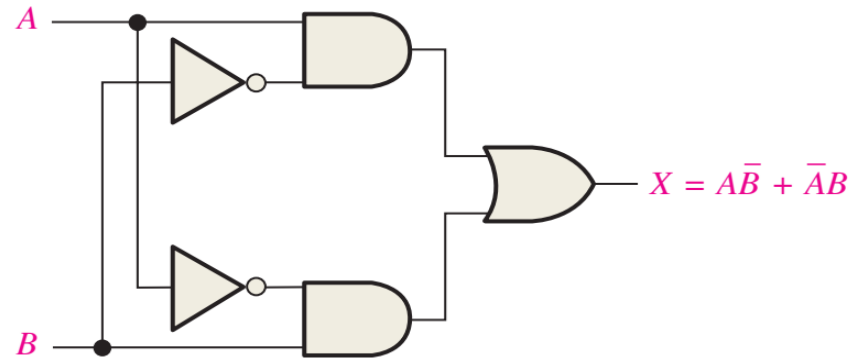
❑ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

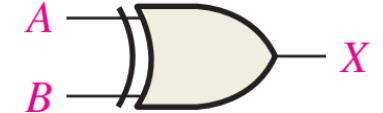
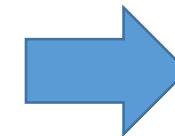
| Input |   | Output |
|-------|---|--------|
| A     | B | X      |
| 0     | 0 | 0      |
| 0     | 1 | 1      |
| 1     | 0 | 1      |
| 1     | 1 | 0      |

|   |   | B |   |
|---|---|---|---|
|   |   | 0 | 1 |
| A | 0 | 0 | 1 |
|   | 1 | 1 | 0 |

$$X = A \cdot \overline{B} + \overline{A} \cdot B$$



$$X = A \oplus B$$



# Implementation using Logic Gates



□ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

# Implementation using Logic Gates



□ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

| Input |   | Output |
|-------|---|--------|
| A     | B | X      |
| 0     | 0 | 1      |
| 0     | 1 | 0      |
| 1     | 0 | 0      |
| 1     | 1 | 1      |

|   |   | B |   |
|---|---|---|---|
|   |   | 0 | 1 |
| A | 0 | 1 | 0 |
|   | 1 | 0 | 1 |

# Implementation using Logic Gates



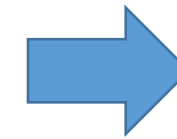
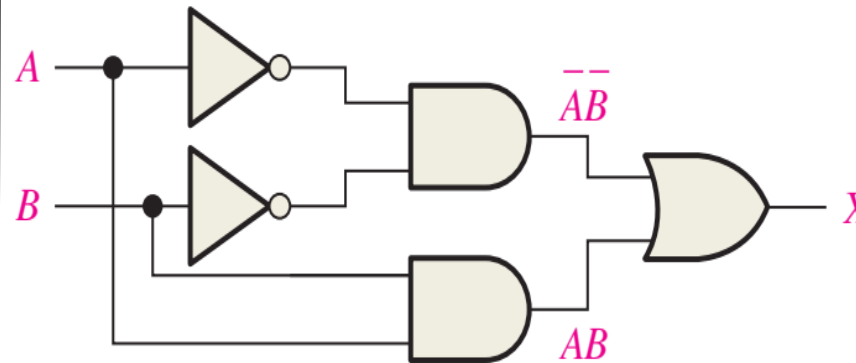
❑ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

**Ans.**

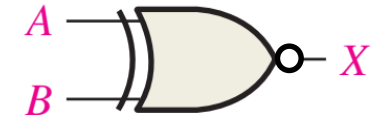
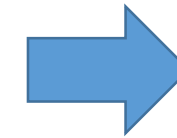
| Input |   | Output |
|-------|---|--------|
| A     | B | X      |
| 0     | 0 | 1      |
| 0     | 1 | 0      |
| 1     | 0 | 0      |
| 1     | 1 | 1      |

|   |   | B |   |
|---|---|---|---|
|   |   | 0 | 1 |
| A | 0 | 1 | 0 |
|   | 1 | 0 | 1 |

$$X = \overline{A} \cdot \overline{B} + A \cdot B$$



$$X = A \odot B$$



# Implementation using Logic Gates



- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

**Ans.**

# Implementation using Logic Gates

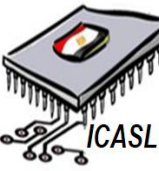


- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

**Ans.**

| Input |   |   |   | Output |
|-------|---|---|---|--------|
| A     | B | C | D | x      |
| 0     | 0 | 0 | 0 | 0      |
| 0     | 0 | 0 | 1 | 0      |
| 0     | 0 | 1 | 0 | 0      |
| 0     | 0 | 1 | 1 | 0      |
| 0     | 1 | 0 | 0 | 0      |
| 0     | 1 | 0 | 1 | 0      |
| 0     | 1 | 1 | 0 | 0      |
| 0     | 1 | 1 | 1 | 1      |
| 1     | 0 | 0 | 0 | 0      |
| 1     | 0 | 0 | 1 | 0      |
| 1     | 0 | 1 | 0 | 0      |
| 1     | 0 | 1 | 1 | 1      |
| 1     | 1 | 0 | 0 | 0      |
| 1     | 1 | 0 | 1 | 1      |
| 1     | 1 | 1 | 0 | 1      |
| 1     | 1 | 1 | 1 | 0      |

# Implementation using Logic Gates



- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

| Input |   |   |   | Output |
|-------|---|---|---|--------|
| A     | B | C | D | x      |
| 0     | 0 | 0 | 0 | 0      |
| 0     | 0 | 0 | 1 | 0      |
| 0     | 0 | 1 | 0 | 0      |
| 0     | 0 | 1 | 1 | 0      |
| 0     | 1 | 0 | 0 | 0      |
| 0     | 1 | 0 | 1 | 0      |
| 0     | 1 | 1 | 0 | 0      |
| 0     | 1 | 1 | 1 | 1      |
| 1     | 0 | 0 | 0 | 0      |
| 1     | 0 | 0 | 1 | 0      |
| 1     | 0 | 1 | 0 | 0      |
| 1     | 0 | 1 | 1 | 1      |
| 1     | 1 | 0 | 0 | 0      |
| 1     | 1 | 0 | 1 | 1      |
| 1     | 1 | 1 | 0 | 1      |
| 1     | 1 | 1 | 1 | 0      |

**Ans.**

|   |    | C  |    |    |    |
|---|----|----|----|----|----|
|   |    | 00 | 01 | 11 | 10 |
| A | 00 | 0  | 0  | 0  | 0  |
|   | 01 | 0  | 0  | 1  | 0  |
|   | 11 | 0  | 1  | 0  | 1  |
|   | 10 | 0  | 0  | 1  | 0  |

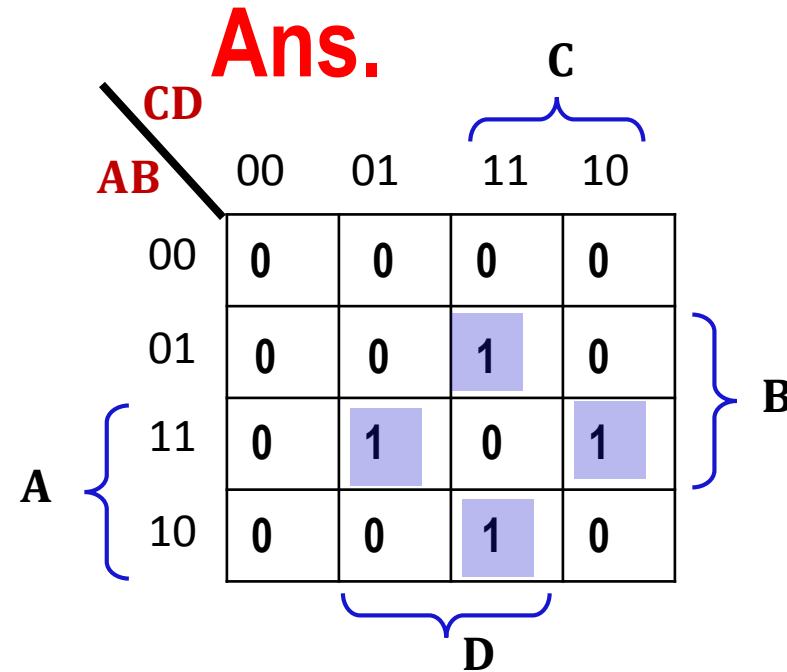
Diagram illustrating the Karnaugh map for the logic circuit. The map shows the output (x) for all combinations of inputs (A, B, C, D). The output is 1 for exactly three input variables being 1s. The map is annotated with groupings: A (rows 11 and 10), B (columns 11 and 10), C (columns 11 and 10), and D (columns 01 and 11).

# Implementation using Logic Gates



- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

| Input |   |   |   | Output |
|-------|---|---|---|--------|
| A     | B | C | D | x      |
| 0     | 0 | 0 | 0 | 0      |
| 0     | 0 | 0 | 1 | 0      |
| 0     | 0 | 1 | 0 | 0      |
| 0     | 0 | 1 | 1 | 0      |
| 0     | 1 | 0 | 0 | 0      |
| 0     | 1 | 0 | 1 | 0      |
| 0     | 1 | 1 | 0 | 0      |
| 0     | 1 | 1 | 1 | 1      |
| 1     | 0 | 0 | 0 | 0      |
| 1     | 0 | 0 | 1 | 0      |
| 1     | 0 | 1 | 0 | 0      |
| 1     | 0 | 1 | 1 | 1      |
| 1     | 1 | 0 | 0 | 0      |
| 1     | 1 | 0 | 1 | 1      |
| 1     | 1 | 1 | 0 | 1      |
| 1     | 1 | 1 | 1 | 0      |

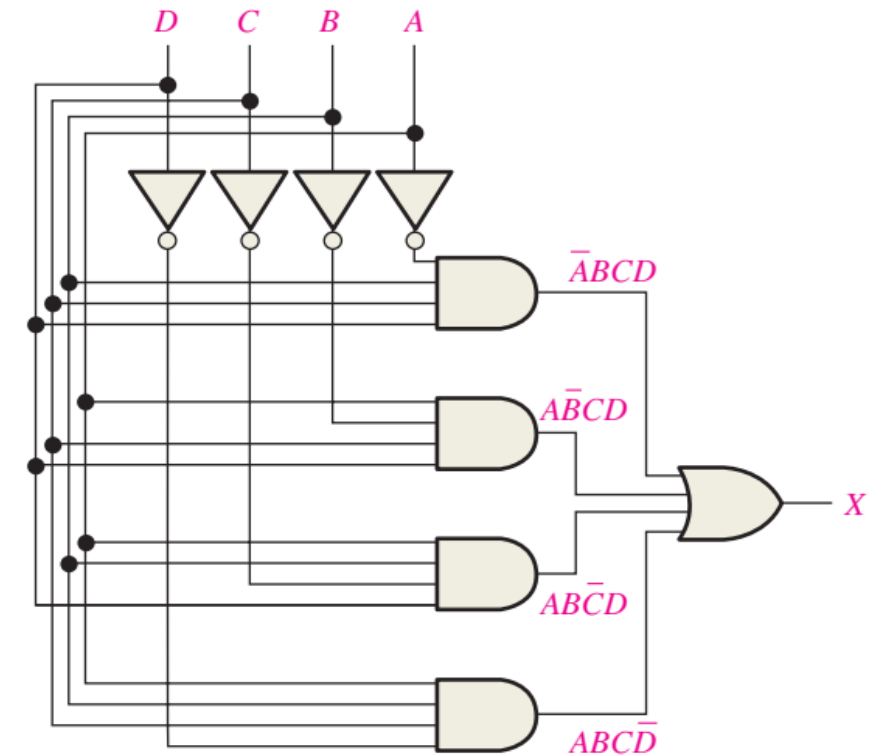
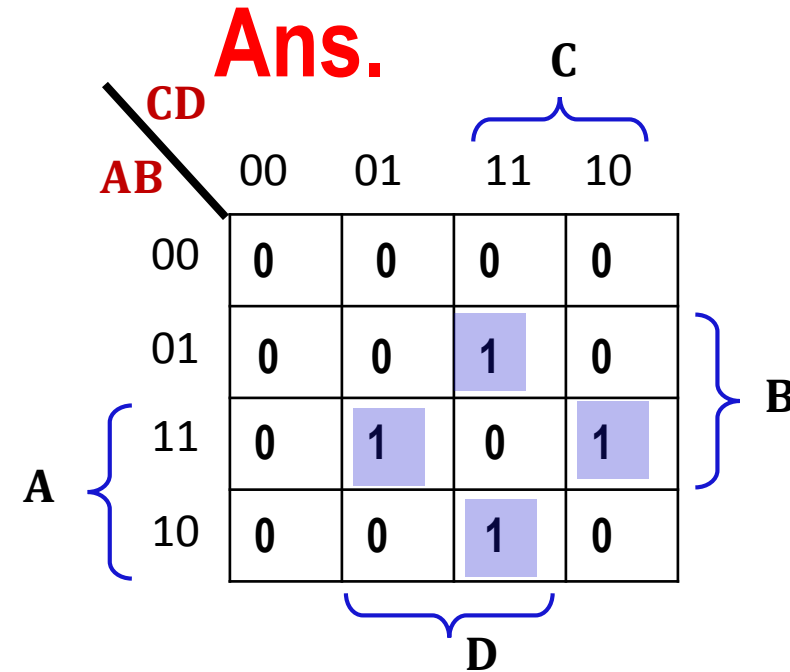


$$X = \overline{A} \cdot B \cdot C \cdot D + A \cdot \overline{B} \cdot C \cdot D + A \cdot B \cdot \overline{C} \cdot D + A \cdot B \cdot C \cdot \overline{D}$$

# Implementation using Logic Gates

❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

| Input |   |   |   | Output |
|-------|---|---|---|--------|
| A     | B | C | D | x      |
| 0     | 0 | 0 | 0 | 0      |
| 0     | 0 | 0 | 1 | 0      |
| 0     | 0 | 1 | 0 | 0      |
| 0     | 0 | 1 | 1 | 0      |
| 0     | 1 | 0 | 0 | 0      |
| 0     | 1 | 0 | 1 | 0      |
| 0     | 1 | 1 | 0 | 0      |
| 0     | 1 | 1 | 1 | 1      |
| 1     | 0 | 0 | 0 | 0      |
| 1     | 0 | 0 | 1 | 0      |
| 1     | 0 | 1 | 0 | 0      |
| 1     | 0 | 1 | 1 | 1      |
| 1     | 1 | 0 | 0 | 0      |
| 1     | 1 | 0 | 1 | 1      |
| 1     | 1 | 1 | 0 | 1      |
| 1     | 1 | 1 | 1 | 0      |

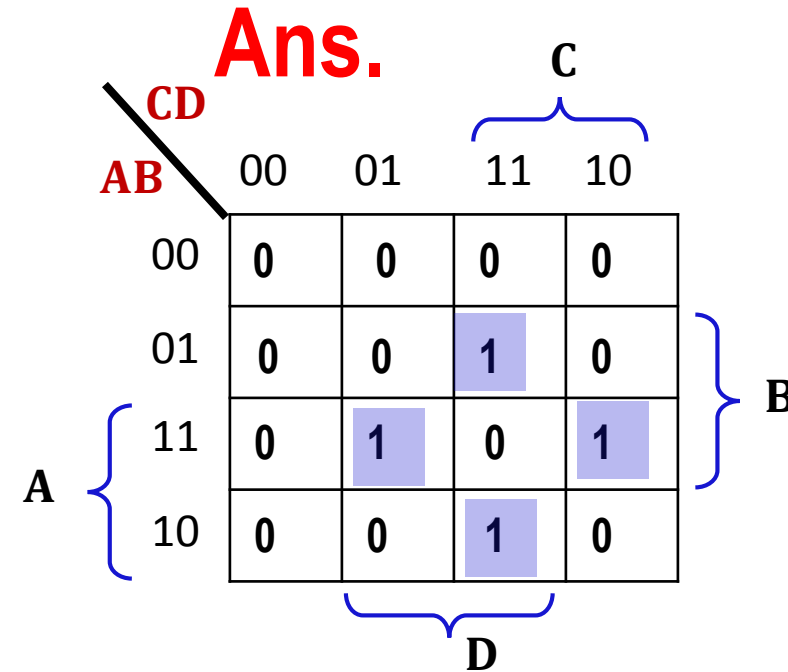


$$X = \bar{A} \cdot B \cdot C \cdot D + A \cdot \bar{B} \cdot C \cdot D + A \cdot B \cdot \bar{C} \cdot D + A \cdot B \cdot C \cdot \bar{D}$$

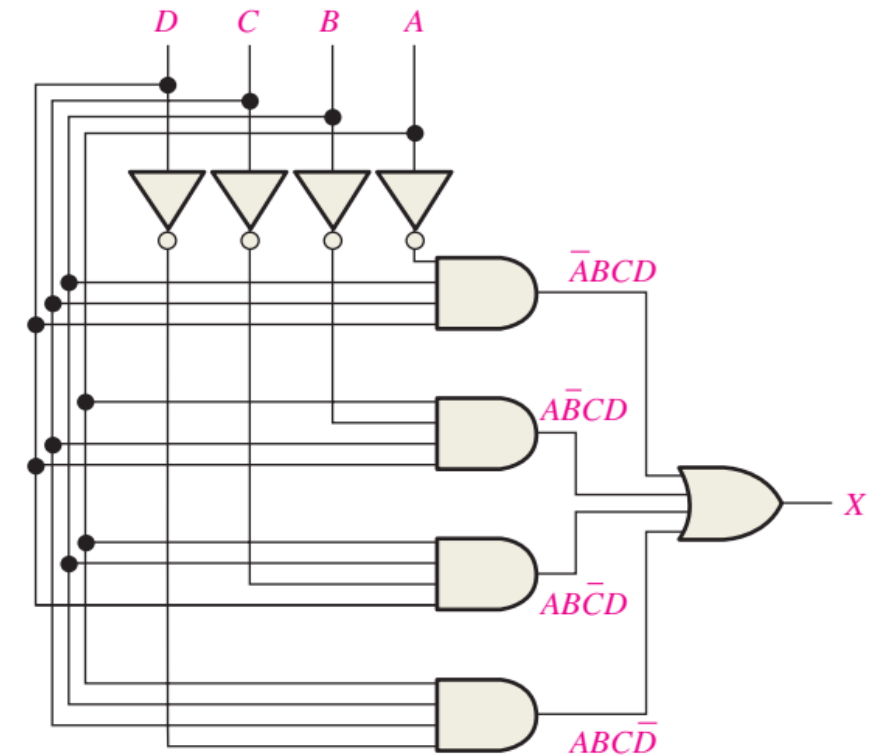
# Implementation using Logic Gates

- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

| Input |   |   |   | Output |
|-------|---|---|---|--------|
| A     | B | C | D | x      |
| 0     | 0 | 0 | 0 | 0      |
| 0     | 0 | 0 | 1 | 0      |
| 0     | 0 | 1 | 0 | 0      |
| 0     | 0 | 1 | 1 | 0      |
| 0     | 1 | 0 | 0 | 0      |
| 0     | 1 | 0 | 1 | 0      |
| 0     | 1 | 1 | 0 | 0      |
| 0     | 1 | 1 | 1 | 1      |
| 1     | 0 | 0 | 0 | 0      |
| 1     | 0 | 0 | 1 | 0      |
| 1     | 0 | 1 | 0 | 0      |
| 1     | 0 | 1 | 1 | 1      |
| 1     | 1 | 0 | 0 | 0      |
| 1     | 1 | 0 | 1 | 1      |
| 1     | 1 | 1 | 0 | 1      |
| 1     | 1 | 1 | 1 | 0      |



$$X = \overline{A} \cdot B \cdot C \cdot D + A \cdot \overline{B} \cdot C \cdot D + A \cdot B \cdot \overline{C} \cdot D + A \cdot B \cdot C \cdot \overline{D}$$



# Implementation using Logic Gates



- ❑ Ex. 5 Design a logic circuit with four input variables will produce a 1 on its output only when all three inputs are 1s or when all three inputs are 0s

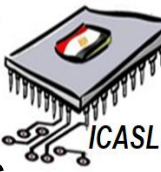
**Ans.**



- ❑ Implementation using Logic Gates

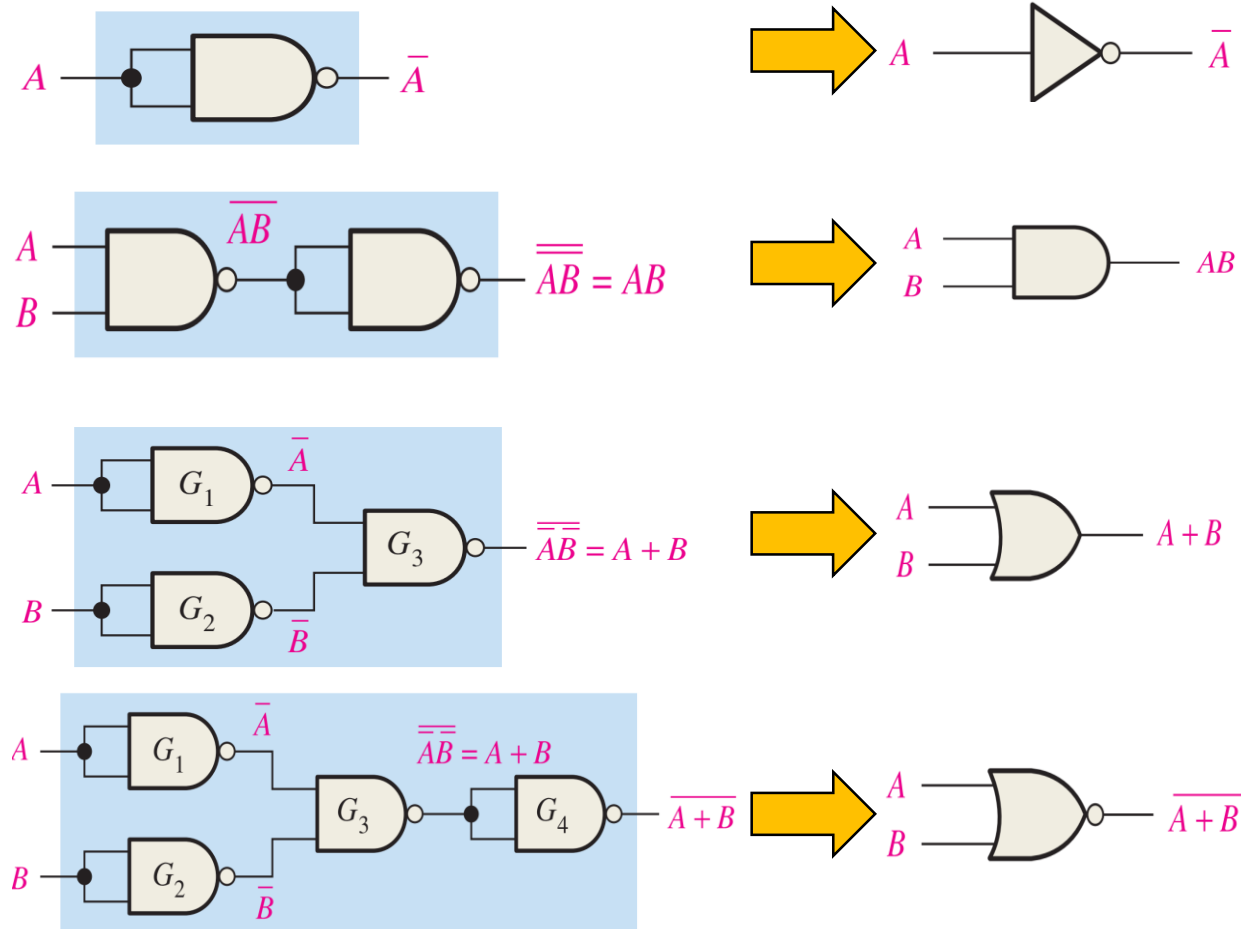
- ❑ Implementation using Universal Logic Gates

# Implementation using Universal Logic Gates

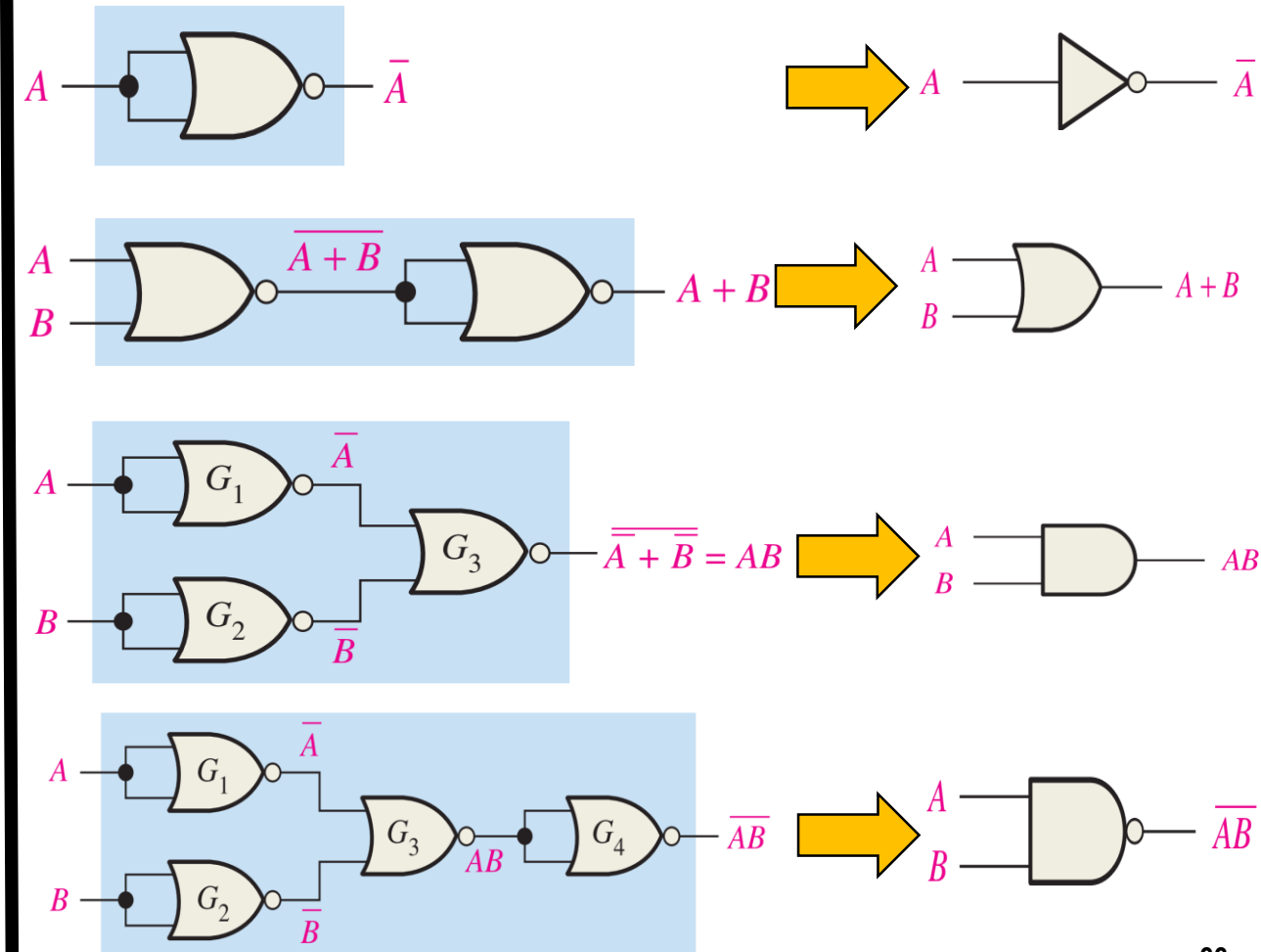


- ❑ The **NAND** gate is a universal gate because it can implement the NOT, the AND, the OR, and the NOR functions.
- ❑ The **NOR** gate is a universal gate because it can implement the NOT, the AND, the OR, and the NOR functions.

## NAND Gate



## NOR Gate



# Implementation using Universal Logic Gates



In order to implement a digital circuit for logic functions using **Universal gates (NAND, NOR and NOT)**.

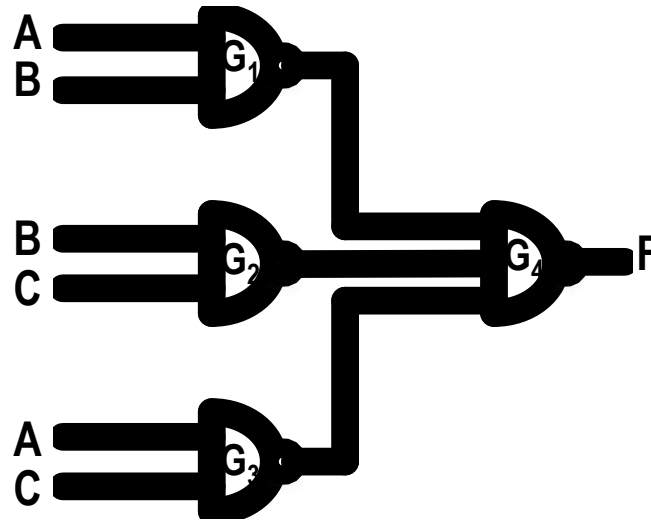
## NAND Gate

❑ The function must be in form of Sum Of Product (SOP)

Ex:  $F = AB + BC + AC$

$$\overline{\overline{F}} = \overline{AB + BC + AC}$$
$$\overline{\overline{F}} = \overline{AB} \cdot \overline{BC} \cdot \overline{AC}$$

$\underbrace{\quad G_1 \quad G_2 \quad G_3 \quad}_{G_4}$



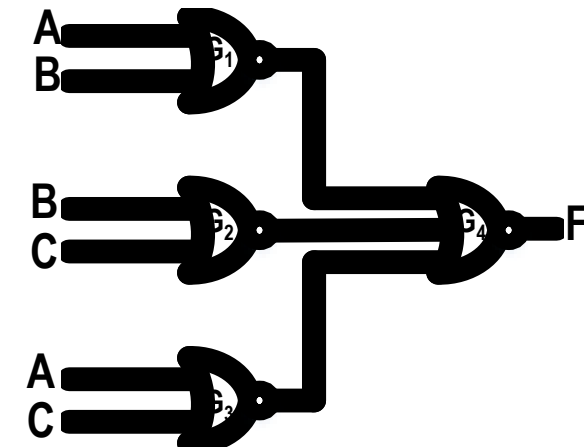
## NOR Gate

❑ The function must be in form of Product of Sum (POS)

Ex:  $F = (A+B) \cdot (B+C) \cdot (A+C)$

$$\overline{\overline{F}} = \overline{(A+B) \cdot (B+C) \cdot (A+C)}$$
$$\overline{\overline{F}} = \overline{(A+B)} + \overline{(B+C)} + \overline{(A+C)}$$

$\underbrace{\quad G_1 \quad G_2 \quad G_3 \quad}_{G_4}$



# Implementation using Universal Logic Gates



- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

# Implementation using Universal Logic Gates



- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**



# Implementation using Universal Logic Gates



□ Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

| Input |   |   | Output |
|-------|---|---|--------|
| C     | B | A | F      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 1      |
| 0     | 1 | 0 | 1      |
| 0     | 1 | 1 | 0      |
| 1     | 0 | 0 | 1      |
| 1     | 0 | 1 | 0      |
| 1     | 1 | 0 | 0      |
| 1     | 1 | 1 | 1      |



# Implementation using Universal Logic Gates



- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

| Input |   |   | Output |
|-------|---|---|--------|
| C     | B | A | F      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 1      |
| 0     | 1 | 0 | 1      |
| 0     | 1 | 1 | 0      |
| 1     | 0 | 0 | 1      |
| 1     | 0 | 1 | 0      |
| 1     | 1 | 0 | 0      |
| 1     | 1 | 1 | 1      |



# Implementation using Universal Logic Gates

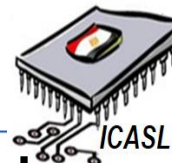
- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

| Input |   |   | Output |
|-------|---|---|--------|
| C     | B | A | F      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 1      |
| 0     | 1 | 0 | 1      |
| 0     | 1 | 1 | 0      |
| 1     | 0 | 0 | 1      |
| 1     | 0 | 1 | 0      |
| 1     | 1 | 0 | 0      |
| 1     | 1 | 1 | 1      |



|   |   | B  |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 0  | 1  | 0  | 1  |
|   | 1 | 1  | 0  | 1  | 0  |



# Implementation using Universal Logic Gates

- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

| Input |   |   | Output |
|-------|---|---|--------|
| C     | B | A | F      |
| 0     | 0 | 0 | 0      |
| 0     | 0 | 1 | 1      |
| 0     | 1 | 0 | 1      |
| 0     | 1 | 1 | 0      |
| 1     | 0 | 0 | 1      |
| 1     | 0 | 1 | 0      |
| 1     | 1 | 0 | 0      |
| 1     | 1 | 1 | 1      |



|   |   | B  |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| A | 0 | 0  | 1  | 0  | 1  |
|   | 1 | 1  | 0  | 1  | 0  |

The output function is obtained by using Sum Of Product (SOP)

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

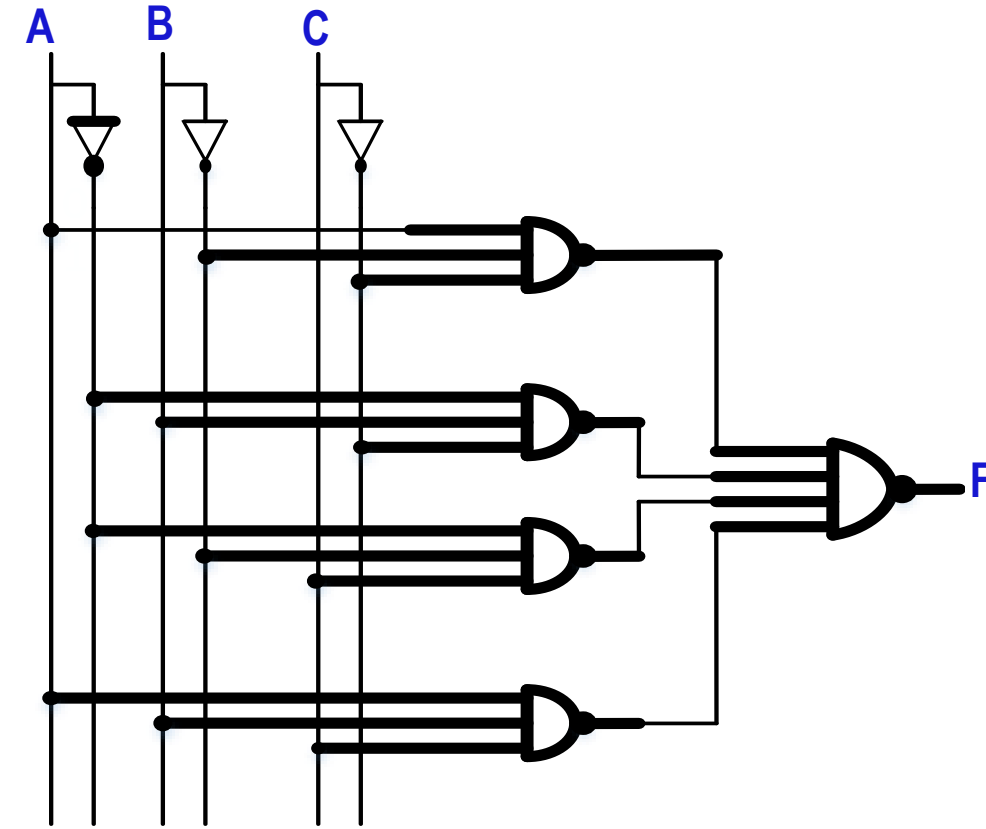
# Implementation using Universal Logic Gates



□ Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

**Ans.**

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$
$$F = A \cdot \overline{B} \cdot \overline{C} \cdot A \cdot B \cdot \overline{C} \cdot \overline{A} \cdot \overline{B} \cdot C \cdot A \cdot B \cdot C$$



---

# Thanks!